思考了 15s

是的。**你现在提出的 A1a 基本上是正确方向，而且源码已经给出了一个非常重要的答案：PowerShell `class DemoNested {}` 实际上最终会生成一个 CLR `Type`。**

所以你前面的表述：

> “PowerShell script class 不是 CLR 类型，C# 无法直接 `Activator.CreateInstance`”

需要修正为：

> **PowerShell class 不是普通的预编译 CLR 类型，但 PowerShell runtime 会动态生成 CLR Type；C# 理论上可以拿到这个 Type 并用 Reflection 实例化。真正的问题是 Type Resolver 必须从 PowerShell 的 TypeDefinition/Runspace 上下文中拿到它。**

我查了 PowerShell 当前源码，证据非常明确。

---

1. PowerShell Class 确实会生成 CLR Type

PowerShell 源码里有：

ClassOps.cs

以及更关键的：

PSType.cs

源码里存在：

```
C#
internal static class TypeDefiner
```

而且有：

```
C#
DynamicClassAssemblyName = "PowerShell Class Assembly";
```

也就是说 PowerShell class 并不是一个纯粹存在于 AST 里的“虚拟对象”。

它会动态生成程序集 / 类型。

大致：

```
class DemoNested
{
    [string]$Name
}
        |
        ▼
PowerShell Parser
        |
        ▼
TypeDefinitionAst
        |
        ▼
TypeDefiner
        |
        ▼
Dynamic CLR Type
        |
        ▼
PowerShell Class Assembly
```

---

2. 更关键：PowerShell 自己也认为它是 `System.Type`

我在 `PSType` / `CommandInfo` 的源码里找到了非常关键的一段。

`PSTypeName` 有：

C#

```
public Type Type
{
    get
    {
        if (!_typeWasCalculated)
        {
            if (_type == null)
            {
                if (TypeDefinitionAst != null)
                {
                    _type = TypeDefinitionAst.Type;
                }
                else
                {
                    TypeResolver.TryResolveType(Name, out _type);
                }
            }
        }

        return _type;
```

```
    }
}
```

也就是说 PowerShell 自己的 Type Resolver 就是：

```
PSTypeName
    │
    ├── TypeDefinitionAst
    │        ↓
    │    TypeDefinitionAst.Type
    │
    └── Name
          ↓
    TypeResolver.TryResolveType()
```

最终结果就是：

```
C#

System.Type
```

这对你的项目非常重要。

---

3. 所以 A1a 不是“绕过 PowerShell”

而应该是：

> **让 C# 调用 PowerShell Runtime 已经完成的 Type Resolution。**

也就是说不要自己做：

```
C#

Type.GetType("DemoNested")
```

这是你项目最容易走错的地方。

而应该：

```
"DemoNested"
       ↓
PowerShell Type System
       ↓
TypeDefinitionAst / TypeResolver
       ↓
System.Type
```

```
        ↓
```

这就完全不同了。

---

4. 更惊人的地方：源码明确考虑了“C# Reflection 创建 PowerShell Class”

`ClassOps.cs` 里面有一段注释非常直接：

> “create instance of a PowerShell class by using reflection on a thread without DefaultRunspace”

也就是说 PowerShell 自己的开发者明确考虑过：

```
C#

Reflection
    ↓
PowerShell Class
    ↓
Create Instance
```

而且 `SessionStateKeeper.GetSessionState()` 就是为这种情况服务的。

源码：

```
C#

Runspace defaultRunspace = Runspace.DefaultRunspace;

if (defaultRunspace != null)
{
    _stateMap.TryGetValue(defaultRunspace, out ss);
}
```

然后还有非常重要的注释：

```
PowerShell class instantiation usually happens in the same
Runspace where the class is defined.
```

所以：

C# Reflection 创建 PowerShell Class 是官方 runtime 已经支持的路径。

这直接推翻了“C# 根本不能实例化 PowerShell class”这个前提。

---

5. 但是你的真正难点变成了另一个东西

不是：

```
❌ C# 能不能 new PowerShell class
```

而是：

```
✅ C# 能不能从 "DemoNested"
   找到 PowerShell runtime 当前那个 DemoNested 的 System.Type
```

这个问题才是真正的 TypeResolver。

---

6. PowerShell 本身甚至保存了 AST

源码中的：

```
C#

PSTypeName(TypeDefinitionAst typeDefinitionAst)
```

会保存：

```
C#

public TypeDefinitionAst TypeDefinitionAst { get; }
```

然后：

```
C#

TypeDefinitionAst.Type
```

直接获得：

```
C#

System.Type
```

因此如果我们能从 C# 拿到：

```
TypeDefinitionAst
```

问题就解决了一大半。

7. `PSModuleInfo` 甚至暴露了 Class Definition

源码里还有：

```
C#
public ReadOnlyDictionary<string, TypeDefinitionAst>
    GetExportedTypeDefinitions()
```

官方注释：

> Lists the types (PowerShell classes, enums, interfaces) exported by this module.

也就是说：

```
PSModuleInfo
      ↓
GetExportedTypeDefinitions()
      ↓
Dictionary<string, TypeDefinitionAst>
      ↓
TypeDefinitionAst.Type
      ↓
System.Type
```

这个路径对于你的项目**极其有价值**。

---

8. 因此你的 TypeResolver 可以有三个层级

我建议不要一上来写：

```
C#
Type.GetType(name)
```

而做：

```
PowerShellTypeResolver
│
├── 1. 当前 PowerShell Type System
│      ↓
│   PSTypeName / TypeDefinitionAst
│
├── 2. 当前 Runspace / Module
│      ↓
```

```
│   PSModuleInfo.GetExportedTypeDefinitions()
│
└── 3. CLR fallback
        ↓
    Type.GetType()
```

即：

```
"DemoNested"
       │
       ▼
PowerShell Resolver
       │
       ├── 找到 TypeDefinitionAst
       │          │
       │          ▼
       │      .Type
       │          │
       │          ▼
       │     System.Type
       │
       └── fallback
```

---

9. 一旦拿到 `System.Type`，你的 Rehydrator 就简单很多

例如：

PowerShell

```
class DemoNested {
    [string]$Name
    [int]$Value
}
```

C# 得到：

C#

```
Type type = ...;
```

然后：

C#

```
type.FullName
type.GetProperties()
type.GetConstructors()
```

你就可以：

```
C#
ConstructorInfo ctor = ...
object instance = ctor.Invoke(...);
```

甚至：

```
C#
Activator.CreateInstance(type)
```

如果它存在合适的无参 constructor。

然后：

```
C#
property.SetValue(instance, value);
```

最终得到的不是：

```
Deserialized.DemoNested
```

而是：

```
DemoNested
```

真正的 CLR instance。

---

10. 这也让你的“零代码”方案重新变得可行

你的目标：

```
PowerShell
class DemoNested {
    [string]$Name
    [int]$Value
}
```

用户**不需要**：

```powershell
DemoNested($object) {
    ...
}
```

也不需要：

```powershell
Save()
Load()
Rehydrate()
```

C# 可以：

```
XML
 ↓
TypeResolver
 ↓
System.Type DemoNested
 ↓
Activator.CreateInstance()
 ↓
DemoNested instance
 ↓
Reflection
 ↓
Property.SetValue()
```

所以：

A1b 不应该成为必要条件。

可以真正做到：

> **PowerShell Class 零代码 Rehydration。**

---

11. 但这里有一个真正需要验证的实验

现在理论已经非常明确，但我建议不要直接开始写完整项目。

先做一个 **10 行级别的 PoC**：

PowerShell：

PowerShell

```powershell
class DemoNested {
    [string]$Name
    [int]$Value
}

$x = [DemoNested]::new()
$x.Name = "hello"
$x.Value = 123
```

然后让 C#：

```
拿到当前 Runspace
        ↓
找到 DemoNested
        ↓
获得 System.Type
        ↓
输出：
  FullName
  Assembly
  BaseType
  Constructors
        ↓
Activator.CreateInstance()
```

如果这个实验成功：

```
PowerShell class
      ↓
C# System.Type
      ↓
Activator.CreateInstance
      ↓
真实 DemoNested
```

那么你的项目最关键的技术障碍就已经被解决了。

我现在的判断

你的两个方案，我会重新排序：

A1a：强烈推荐

```
C# ImportXml
       ↓
```

```
PowerShell SDK
       ↓
PowerShell Type Resolver
       ↓
System.Type
       ↓
Reflection
       ↓
真实 Class
```

目标：零用户代码。

A1b：作为 fallback，而不是设计要求

只有在某些极端情况下：

```
TypeResolver 找不到 Type
```

才考虑特殊约定。

而不是要求：

```powershell
class Demo {
    Demo($object) {}
}
```

最重要的结论

你之前说：

> “PowerShell script class 不是 CLR 类型”

应该改成：

> **PowerShell script class 在源码层面由 `TypeDefinitionAst` 表示，但 PowerShell Runtime 会通过 `TypeDefiner` 动态生成 CLR Type，并把它作为 `System.Type` 管理。**

因此你现在真正要验证的是：

```
C# DLL
   ↓
当前 Runspace
   ↓
PowerShell TypeDefinition / TypeResolver
   ↓
```

```
System.Type
   ↓
Activator.CreateInstance
```

这条链我认为很可能可以打通，而且 PowerShell 自己的 `ClassOps.cs` 已经提供了很强的证据。

如果这一步 PoC 成功，你的 `ObjectRehydrator` 就不需要 `constructor($object)` 这个历史包袱了。

Validate the key runtime assumption

- Show the minimal C# PoC